

Law of Demeter:

1.

- A) In the printSomePoints method we only access the getters of the given parameters. The method calls only on its direct "friends" (point), so it follow the law
- B) The printDistanceBetweenRandomPointsOfTwoSquares method has two parameters (friends): s1 and s2. **Point randomPoint1 = s1.getRandomPoint();** Here, the method talks to its friend s1... but it "asks" for one of s1's internal parts (a Point object). **System.out.println(randomPoint1.distanceTo(randomPoint2));** The method then talks directly to that "friend of a friend" (randomPoint1) by calling its distanceTo method. This is a violation because the printDistance method is reaching through the Square object to get one of its internal components (point) than calls a method on that component.

```
// In the class with the printDistance... method
public void printDistanceBetweenRandomPointsOfTwoSquares(Square s1, Square s2) {
    // "Tell" s1 to calculate the distance, passing s2 as a parameter
    double distance = s1.calculateDistanceToRandomPointOf(s2);
    System.out.println(distance);
}

// ...

// New method inside the Square class
@Data
@AllArgsConstructor
class Square {
    private Point[] points = new Point[4];

    public Point getRandomPoint() {
        int index = (int) (Math.random() * 4);
        return points[index];
    }

    /**
     * This new method follows the Law of Demeter.
     * It only talks to "itself" (this.getRandomPoint())
     * and its parameter "other" (other.getRandomPoint()).
     */
    public double calculateDistanceToRandomPointOf(Square other) {
        Point p1 = this.getRandomPoint(); // Talk to self
        Point p2 = other.getRandomPoint(); // Talk to parameter

        return p1.distanceTo(p2); // Talk to locally-obtained object
    }
}
```

- C) In someMethod we receive as a parameter an object of type Student. Through it we access a getter to get those students faculty, but then we access through another getter that faculty's name and students no. This violates LOD .

```

@Data
@AllArgsConstructor
class Student {
    private String name;
    private Faculty faculty;

    // --- START OF FIX ---

    /**
     * This is a new delegate method.
     * It "tells" the student to get the faculty's name.
     */
    public String getFacultyName() {
        // This is NOT a violation, because Student is talking
        // to its own field (an immediate friend).
        return this.faculty.getName();
    }

    /**
     * This is another new delegate method.
     */
    public int getFacultyStudentNo() {
        return this.faculty.getStudentNo();
    }

    // --- END OF FIX ---
}

@Data
@AllArgsConstructor
class Faculty {
    private List<Student> students;

    public int getStudentNo() { return students.size(); }
    public void removeStudent(Student student) { students.remove(student); }
    public void addStudent(Student student) { students.add(student); }
}

```

```

@Data
@AllArgsConstructor
class Faculty {
    private List<Student> students;

    public int getStudentNo() { return students.size(); }
    public void removeStudent(Student student) { students.remove(student); }
    public void addStudent(Student student) { students.add(student); }
}

```

OR we can receive as parameter an object of type Faculty (in SomeMethod)

- D) This one respects the Law of Demeter because Student interacts directly with StudentBuilder. We have only one relation between those 2 classes.

2.

A data structure is a method /way to store data and organize it. A class combines data and behavior by using it in specific methods. LoD applies specifically to the relations between objects. From what we know, data structures don't provide information about relations between objects, so LoD applies mainly on classes.

S.O.L.I.D.:

1.

```
public static Collection readLinesFromFile(String absoluteFilePath)
```

Preconditions: absoluteFilePath != null –must not be null, must be a text file that can be read line by line, valid path to an existing file, at OS level, the file should be readable(permission)

Postconditions: A Collection should be returned, Empty File=Empty Collection returned, the collection should contain all lines from the specified file in the order they were read, throw exception if any precondition is violated

Invariants: absoluteFilePath – is not modified (being read-only), also the string itself is not modified

Side effects: A file handle (input stream) is opened and subsequently closed; Memory is allocated to store the Collection and all the String objects for the lines read, the operating system might update “last access time” of the file.

2.

It violates The Interface Segregation Principle(ISP). The client should not be forced to depend on methods they don't use . We call IMembership a “fat” interface . I would refactor the code by breaking the interface into smaller (more role-based/specific focused) interfaces.

```

1 // 1. Interface for basic authentication
2 interface IAuthentication {
3     void login(UserCredentials uc);
4     boolean logout();
5 }
6
7 // 2. Interface for managing account details
8 interface IAccountManagement {
9     UserCredentials register(String userName, String password);
10    UserCredentials changePassword(String userName);
11 }
12
13 // 3. Interface for managing payments
14 interface IPaymentManagement {
15     void addPaymentMethod(UserCredentials uc, PaymentMethod pm);
16 }
17
18 // A full-featured user class can implement all three
19 class UserAccount implements IAuthentication, IAccountManagement, IPaymentManagement {
20     // ... implementations ...
21 }
22
23 // A simple guest session might only need authentication
24 class GuestSession implements IAuthentication {
25     // ... implementations for login/logout ...
26 }

```

3.

This violates the Liskov Substitution Principle. The idea is that preconditions can be weaker (accept more inputs) , but postconditions must be stronger (this means that guarantees more, or at least the same).

After analyzing preconditions: The derived 1 accepts everything the base does (negative numbers) + even numbers. This part is correct. The problem is at postconditions.

After analyzing postconditions: The derived postcondition is weaker than base's. Base contract guarantees receiving a valid, NON-NULL string of lowercase digits. The derived method breaks this contract by potentially returning null (which the client is not prepared to handle, it will cause an Exception). Practically the derived postcondition is weaker than the base one(it needs to be “mai smechera” stronger).

4.

ProductsOffer class is not designed well. It violates the Open/Closed Principle (OCP). Classes/modules should be open for extinction but closed for modification. In this example, every time the customer needs a new filter combination (e.g. name and color, etc.) you should modify the class by adding a new method. This makes the class hard to maintain and leads to many combinational methods, when they are required.

In order to refactor, the ProductsOffer class should not be modified. We should add a filter() method that gets a specification and will use it to decide what product returns.

```

// 2. Create the generic Specification interface
// Definim o interfață pe care toate regulile noastre de filtrare trebuie să o respecte. O regulă
// trebuie să poată răspundă la o singură întrebare: "Acest produs îndeplinește condiția ta?"
interface ISpecification<T> {
    boolean isSatisfiedBy(T item);
}

//Parte deschisa a principiului
// 3. Create concrete (leaf) specifications
// adaugam reguli simple generale
class ColorSpecification implements ISpecification<Product> {
    private String color;
    public ColorSpecification(String color) { this.color = color; }
    @Override
    public boolean isSatisfiedBy(Product item) {
        return item.color.equals(this.color);
    }
}

class WeightSpecification implements ISpecification<Product> {
    private double weight;
    public WeightSpecification(double weight) { this.weight = weight; }
    @Override
    public boolean isSatisfiedBy(Product item) {
        return item.weight == this.weight;
    }
}

// ... other specs like NameSpecification, DateSpecification ...

```

```

// 4. Create composite specifications
// adaugam reguli combinate (extensia, combinatia intre e.g. name and color)
class AndSpecification<T> implements ISpecification<T> {
    private ISpecification<T> left;
    private ISpecification<T> right;
    public AndSpecification(ISpecification<T> left, ISpecification<T> right) {
        this.left = left;
        this.right = right;
    }
    @Override
    public boolean isSatisfiedBy(T item) {
        return left.isSatisfiedBy(item) && right.isSatisfiedBy(item);
    }
}
// ... you can also create OrSpecification, NotSpecification ...

// 5. Refactor the ProductsOffer class
//stergem toate metodele de filtrare si inlocuim cu 1 singura metoda de filtrare generala.
// Partea inchisa a principiului ,modificata doar in aceasta situatie .
class ProductsOffer {
    private List<Product> products = new ArrayList<>();

    // The *only* filter method needed. It's open to extension!
    public List<Product> filter(ISpecification<Product> spec) {
        List<Product> result = new ArrayList<>();
        for (Product p : products) {
            if (spec.isSatisfiedBy(p)) {
                result.add(p);
            }
        }
        return result;
    }

    // ... methods to add products, etc. ...
}

```

5.

The example violates The Liskov Substitution Principle (LSP). The abstract class Vehicle creates a faulty abstraction. It forces all Vehicle subclasses to implement, for example: `pressClutch()` and `getGasLevel()`. 😊 A tesla does not have a "gas level" and not every vehicle uses a clutch.

In order to refactor, we need to create better, more specific abstractions. Don't force behaviors onto classes that don't support them. Use interfaces to add optional behaviors.

```

1  // Base class with truly common properties
2  abstract class Vehicle {
3      private int peopleCapacity;
4      abstract double getSpeed();
5  }
6
7  // Interface for vehicles that have a clutch (manual transmission)
8  interface IManualTransmission {
9      void pressClutch();
10 }
11 // Interface for vehicles that use fuel
12 interface IGasPowered {
13     double getGasLevel();
14 }
15 // Interface for electric vehicles
16 interface IElectric {
17     double getAutonomy(); // Battery level
18 }
19 // Interface for vehicles with storage
20 interface IWithTrunk {
21     int getTrunkCapacity();
22 }
23 interface IWithFrunk {
24     int getFrunkCapacity();
25 }
26 // Now, compose the classes from the behaviors they actually have
27 class Logan extends Vehicle implements IGasPowered, IManualTransmission, IWithTrunk {
28     // ... all methods implemented meaningfully ...
29 }
30 class Tesla extends Vehicle implements IElectric, IWithTrunk, IWithFrunk {
31     // Implements getAutonomy(), no getGasLevel() or pressClutch()
32 }
33 class Scooter extends Vehicle implements IGasPowered {
34     // A scooter (assuming gas) but no clutch
35     // If it's an electric scooter, it would implement IElectric instead
36 }

```

6.

In this example, the Single Responsibility Principle (SRP) is violated. A class should have only 1 reason to change. Our Student class clearly has multiple, unrelated responsibilities. For example, if you need to change how a mean grade is calculated, you must change this class. Also, if you want to change from SQL to NoSQL, you also must change this class.

In order to refactor, you should separate these responsibilities into different classes. One for data and logic operations (logic to calculate mean grade, logic to build a profile) and one for saving/loading Student objects (logic to connect to a database, logic to load a student from the database).

```

1  // 1. The Student class (Domain Object)
2  // Only responsible for student data and business logic.
3  class Student {
4      // ... properties like name, grades, etc.
5
6      public Double calculateMeanGrade() {
7          // ... logic to calculate mean ...
8      }
9
10     public Profile getStudentProfile() {
11         // ... logic to build a profile ...
12     }
13 }
14
15 // 2. The Student Repository (Persistence)
16 // Only responsible for saving/loading Student objects.
17 class StudentRepository {
18
19     public void saveStudent(Student student) {
20         // ... logic to connect to database (SQL, API, etc.) ...
21         // ... and save the student's data ...
22     }
23
24     public Student getStudentById(int id) {
25         // ... logic to load a student from the database ...
26     }
27 }

```

